HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN

BÁO CÁO BÀI THỰC HÀNH
HỌC PHẦN: THỰC TẬP CƠ SỞ
MÃ HỌC PHẦN: INT13147

BÀI THỰC HÀNH 4.1
LẬP TRÌNH CLIENT/SERVER ĐỂ TRAO ĐỔI THÔNG TIN AN TOÀN

Sinh viên thực hiện:

B22DCAT303 - Nguyễn Khắc Trí

Giảng viên hướng dẫn: PGS. TS. Hoàng Xuân Dậu

HỌC KỲ 2 NĂM HỌC 2024-2025

MỤC LỤC

DANH MỤC CÁC HÌNH VẼ 3
DANH MỤC CÁC TỪ VIẾT TẮT 4
CHƯƠNG 1. GIỚI THIỆU CHUNG VỀ BÀI THỰC HÀNH 5
1.1 Mục đích 5
1.2 Tìm hiểu lý thuyết 5
1.2.1 Tìm hiểu về các khái niệm liên quan đến lập trình socket với TCP 5
CHƯƠNG 2. NỘI DUNG THỰC HÀNH 9
2.1 Chuẩn bị môi trường 9
2.2 Các bước thực hiện 9
2.2.1 Lập trình client và server với TCP socket 9
2.2.2 Trao đổi thông điệp giữa client và server và đảm bảo tính toàn vẹn của thông điệp khi trao đổi 11
TÀI LIỆU THAM KHẢO 18

DANH MỤC CÁC HÌNH VẼ

Hình 1 Socket 5
Hình 2 Chức năng của socket 6
Hình 3 Lập trình socket 7
Hình 4 Lập trình client 9
Hình 5 Lập trình server 9
Hình 6 Phía Server sau khi nhận phản hồi từ Client 10
Hình 7 Phía Client sau khi nhận phản hồi từ Server 10
Hình 8 Bắt thông tin được gửi từ Server đến Client 11
Hình 9 Bắt thông tin được gửi từ Client đến Server 11
Hình 10 Sửa đổi chương trình bên Server 12
Hình 11 Sửa đổi chương trình bên Client 12
Hình 12 Kết quả trả về bên Server 12
Hình 13 Kết quả khi chạy chương trình bên Client 13
Hình 14 Client gửi thông điệp bản rõ đến Server 13
Hình 15 Client gửi thông điệp + key đã được mã hóa SHA512 13
Hình 16 Server gửi phản hồi đến Client sau khi nhận thông điệp + key không thay đổi ... 14
Hình 17 Thay đổi key ở Client → Thông điệp mã hash thay đổi 14
Hình 18 Bên Server giữ nguyên 15
Hình 19 Bên Server: mã hash của thông điệp đã bị thay đổi → Không toàn vẹn dữ liệu .. 15
Hình 20 Kết quả, bên Client nhận được thông báo rằng dữ liệu đã bị mất mát 15
Hình 21 Thông điệp gửi từ Client đến Server 16
Hình 22 Khi truyền từ Client → Server, mã hash đã bị thay đổi 16
Hình 23 Server phản hồi Client: Dữ liệu đã bị mất mát 17

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh/Giải thích	Thuật ngữ tiếng Việt/Giải thích
API	Application Programming Interface	Giao diện lập trình ứng dụng
TCP	Transmission Control Protocol	Giao thức điều khiển truyền vận
UDP	User Datagram Protocol	Giao thức dữ liệu người dùng
IP	Internet Protocol	Giao thức Internet
OSI	Open Systems Interconnection	Mô hình kết nối hệ thống mở
MAC	Media Access Control	Kiểm soát truy cập phương tiện
LAN	Local Area Network	Mạng cục bộ
WAN	Wide Area Network	Mạng diện rộng

CHƯƠNG 1. GIỚI THIỆU CHUNG VỀ BÀI THỰC HÀNH

1.1 Mục đích

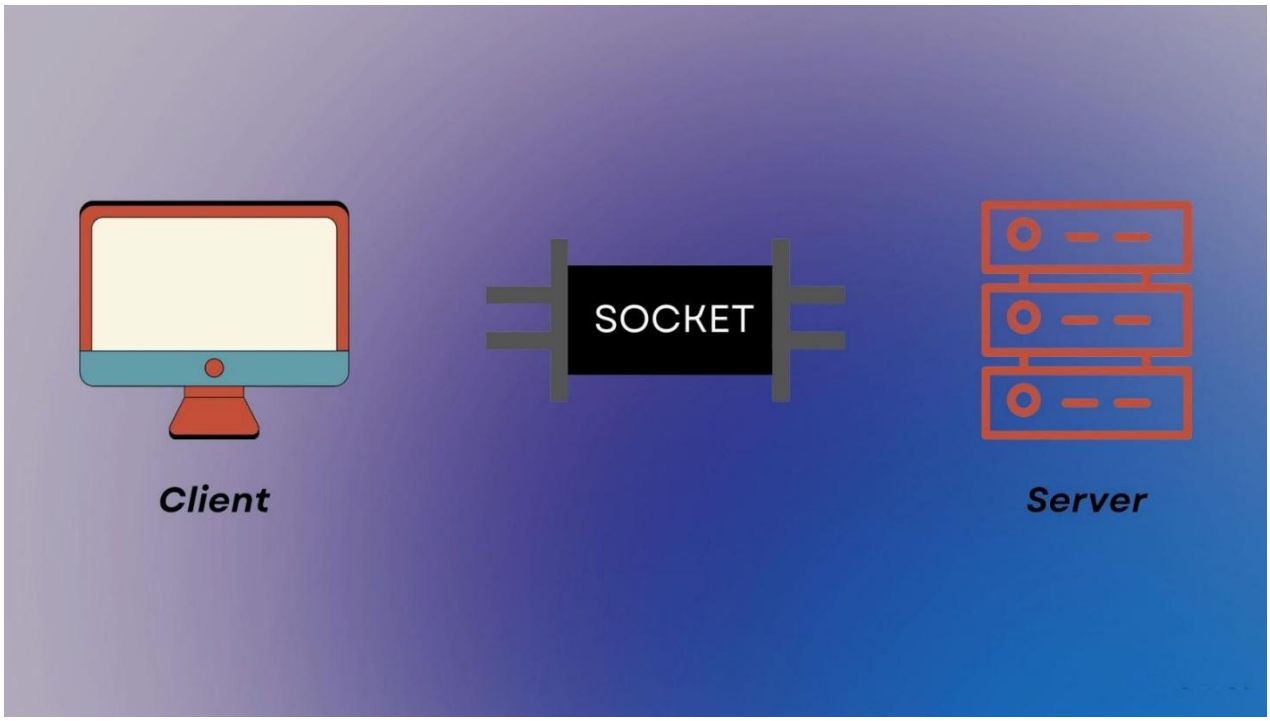
- Sinh viên hiểu về cơ chế client/server và có thể tự lập trình client/server dựa trên socket, sau đó thực hiện ca đặt giao thức đơn giản để trao đổi thông tin an toàn.

1.2 Tìm hiểu lý thuyết

1.2.1 Tìm hiểu về các khái niệm liên quan đến lập trình socket với TCP

1.2.1.1 Socket là gì?

Socket là một điểm cuối (endpoint) trong liên kết truyền thông hai chiều giữa hai tiến trình (process), thường được sử dụng để kết nối giữa client và server trong các ứng dụng mạng.



Hình 1 Socket

- Mỗi socket được liên kết với một địa chỉ IP và một cổng (port) cụ thể.
- Socket hoạt động dựa trên các giao thức như TCP hoặc UDP, cho phép gửi và nhận dữ liệu qua mạng.
- Nó đóng vai trò như giao diện lập trình ứng dụng (API) giữa tầng ứng dụng và tầng vận chuyển (transport layer) của mô hình TCP/IP.
- Giữa hai chương trình cần trao đổi dữ liệu qua mạng, socket là cánh cửa để thiết lập và duy trì kết nối truyền thông hai chiều.

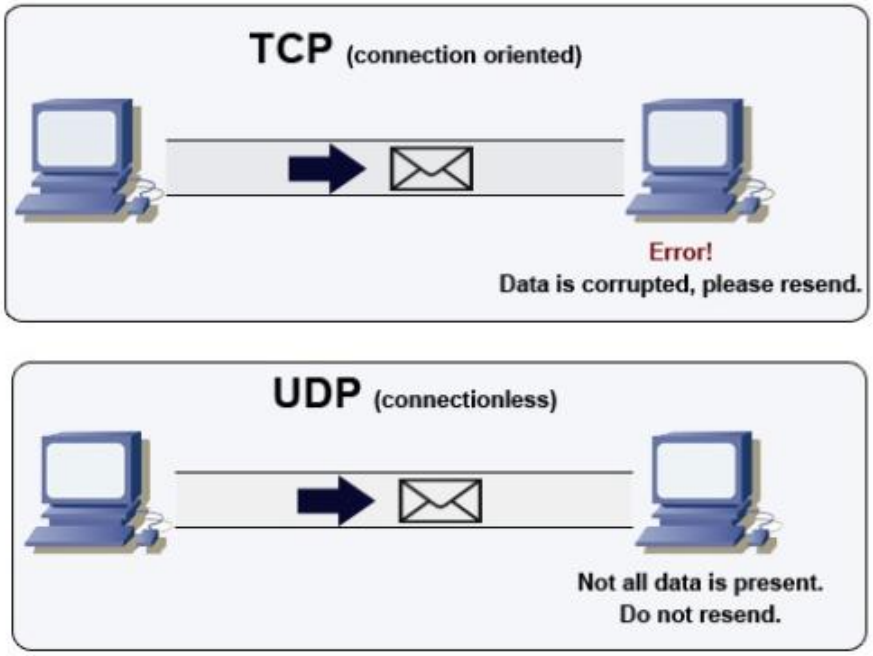
1.2.1.2 Tại sao cần sử dụng Socket?

Socket được sử dụng trong hầu hết các ứng dụng mạng hiện nay vì:

- Kết nối giữa các thiết bị từ xa: ví dụ như chat, truyền tệp, game online.

- Hỗ trợ giao tiếp giữa 2 ứng dụng: chúng có thể nằm trên cùng một máy hoặc trên 2 máy khác nhau.
- Tính linh hoạt: Socket hỗ trợ TCP, UDP và các giao thức khác.
- Phân phối dữ liệu dễ dàng: như truyền video, âm thanh hoặc dữ liệu đa người dùng.
- Tương thích đa nền tảng: Socket có thể chạy trên Windows, Linux, macOS, Android, iOS,...
- Kiểm soát độ trễ: Phù hợp với ứng dụng yêu cầu thời gian thực như VoIP, game,...

1.2.1.3 Điều kiện để Socket hoạt động



Hình 2 Chức năng của socket

Socket chỉ hoạt động khi có đủ thông tin kết nối:

- Địa chỉ IP của thiết bị đích.
- Số hiệu cổng (port) của ứng dụng ở cả client và server.
- Không trùng cổng nếu nhiều socket cùng chạy trên một máy.
- Phải sử dụng giao thức mạng hợp lệ như TCP/IP hoặc UDP/IP.

1.2.1.4 Phân loại Socket

Theo giao thức:

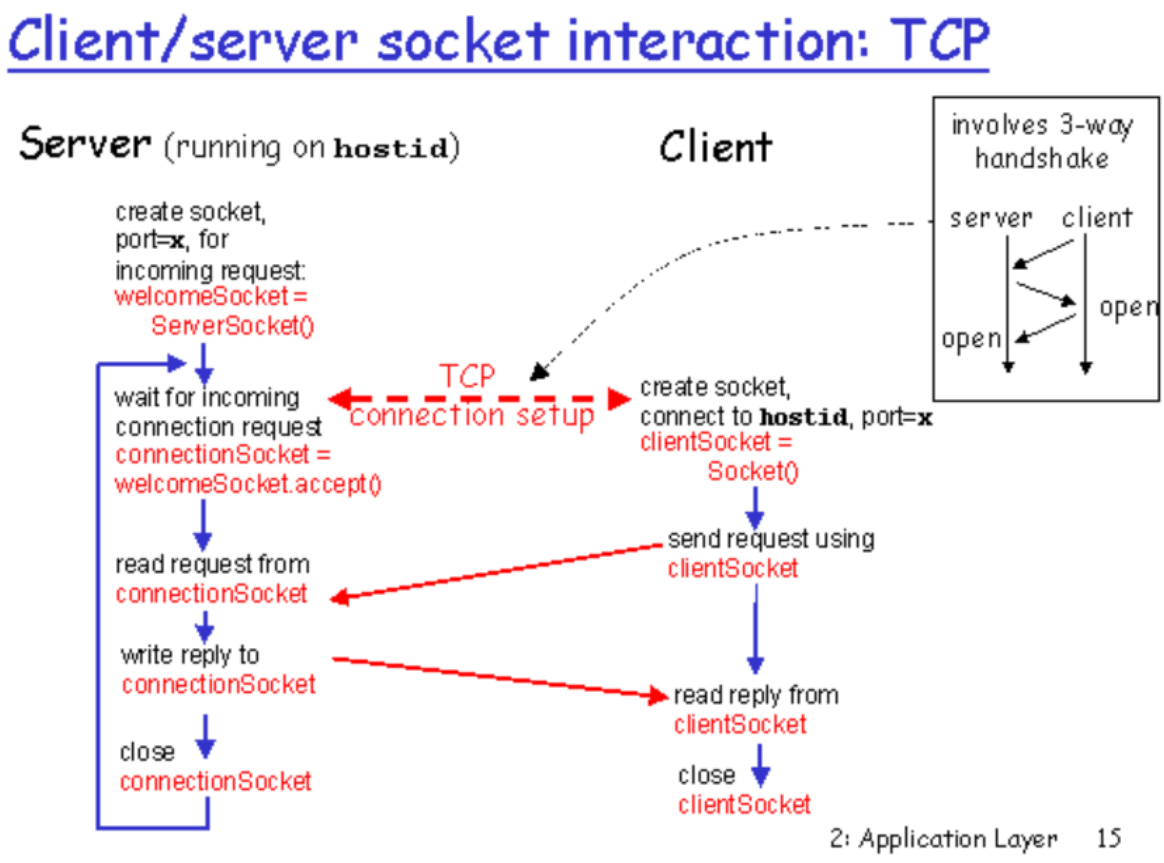
- TCP Socket: Đảm bảo truyền dữ liệu chính xác, có kiểm soát lỗi. Dùng trong email, web, file transfer,...
- UDP Socket: Truyền nhanh, không đảm bảo toàn vẹn. Dùng trong video streaming, game real-time,...

Theo vai trò:

- Server Socket: Dùng để lắng nghe (listen) và chấp nhận kết nối từ client.
- Client Socket: Dùng để gửi yêu cầu kết nối đến server.

1.2.1.5 Lập trình Socket với TCP/IP

Để thiết lập kết nối TCP giữa client và server, cần thực hiện các bước:



Hình 3 Lập trình socket

- Bên Server:
  1. Tạo socket: socket()
  2. Gắn socket với địa chỉ IP và port: bind()
  3. Chờ kết nối từ client: listen()
  4. Chấp nhận kết nối: accept()
- Bên Client:
  1. Tạo socket: socket()
  2. Kết nối đến địa chỉ và port của server: connect()
- Khi kết nối đã thiết lập:
  - Hai bên có thể trao đổi dữ liệu: send() và recv() hoặc read()/write()

- Sau khi xong, đóng kết nối: close()

Quá trình bắt tay 3 bước trong TCP (Three-way Handshake)

1. Client gửi gói SYN đến Server.
2. Server trả lời bằng SYN-ACK.
3. Client phản hồi ACK → kết nối được thiết lập.

Khi accept() được gọi, Server sẽ tạo một socket mới để giao tiếp riêng với Client đó.

CHƯƠNG 2. NỘI DUNG THỰC HÀNH

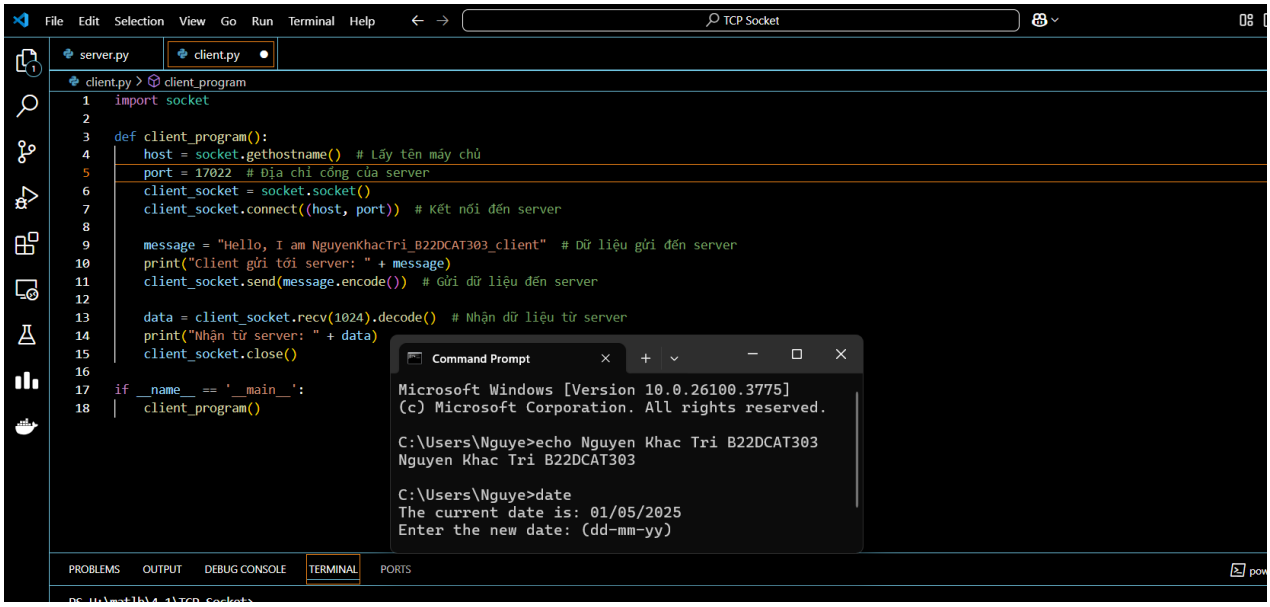
2.1 Chuẩn bị môi trường

- Môi trường Python hoặc Java để chạy được ứng dụng client/server đã lập trình.
- Phần mềm Wireshark

2.2 Các bước thực hiện

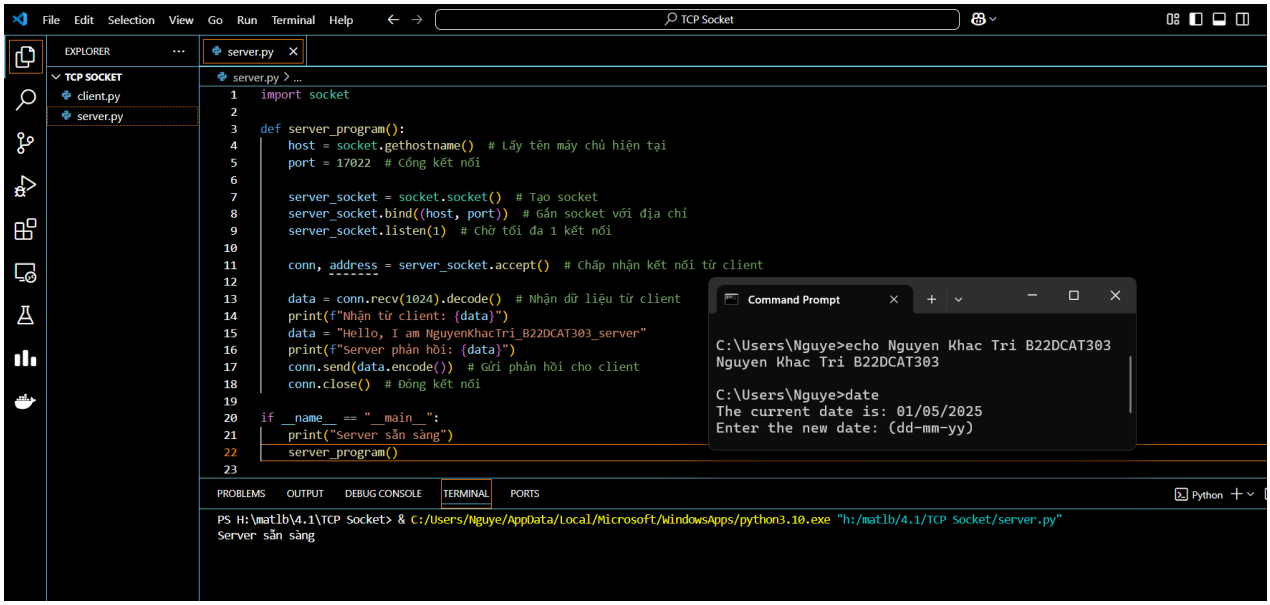
2.2.1 Lập trình client và server với TCP socket

- Lập trình client.



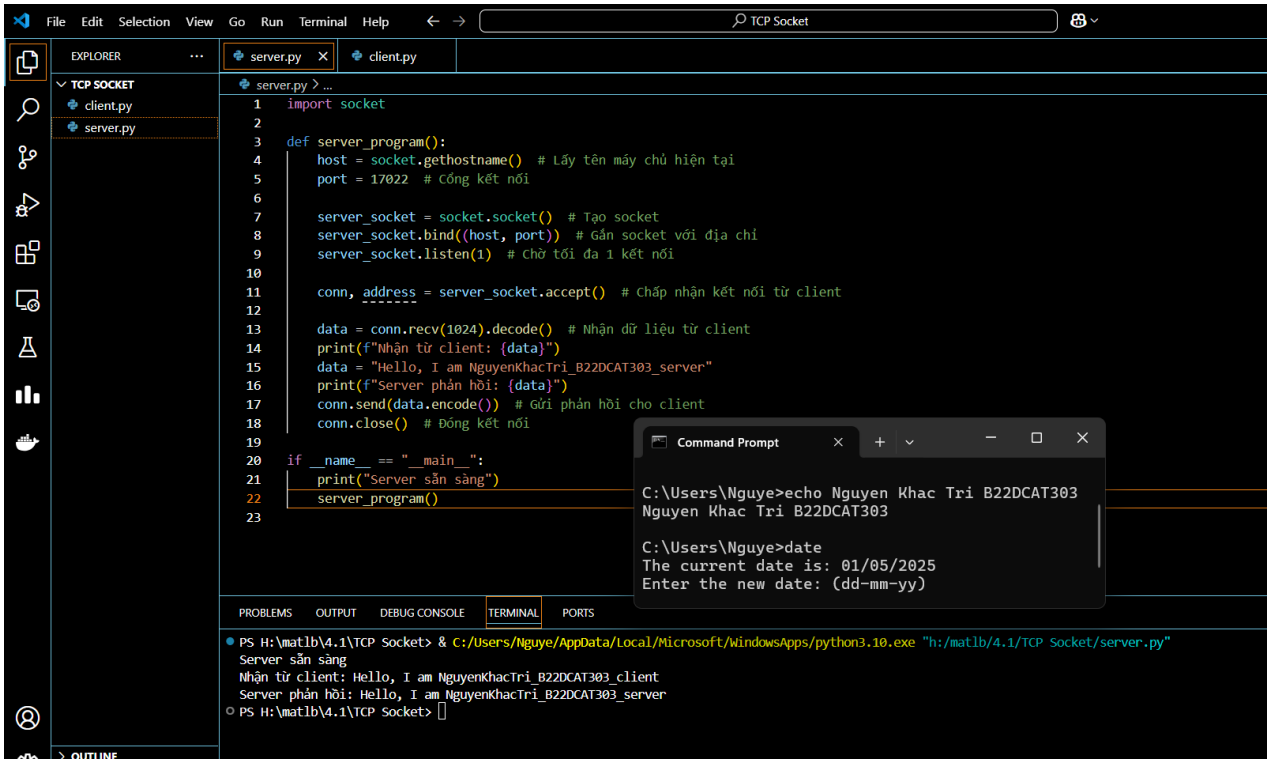
Hình 4 Lập trình client

- Lập trình server.



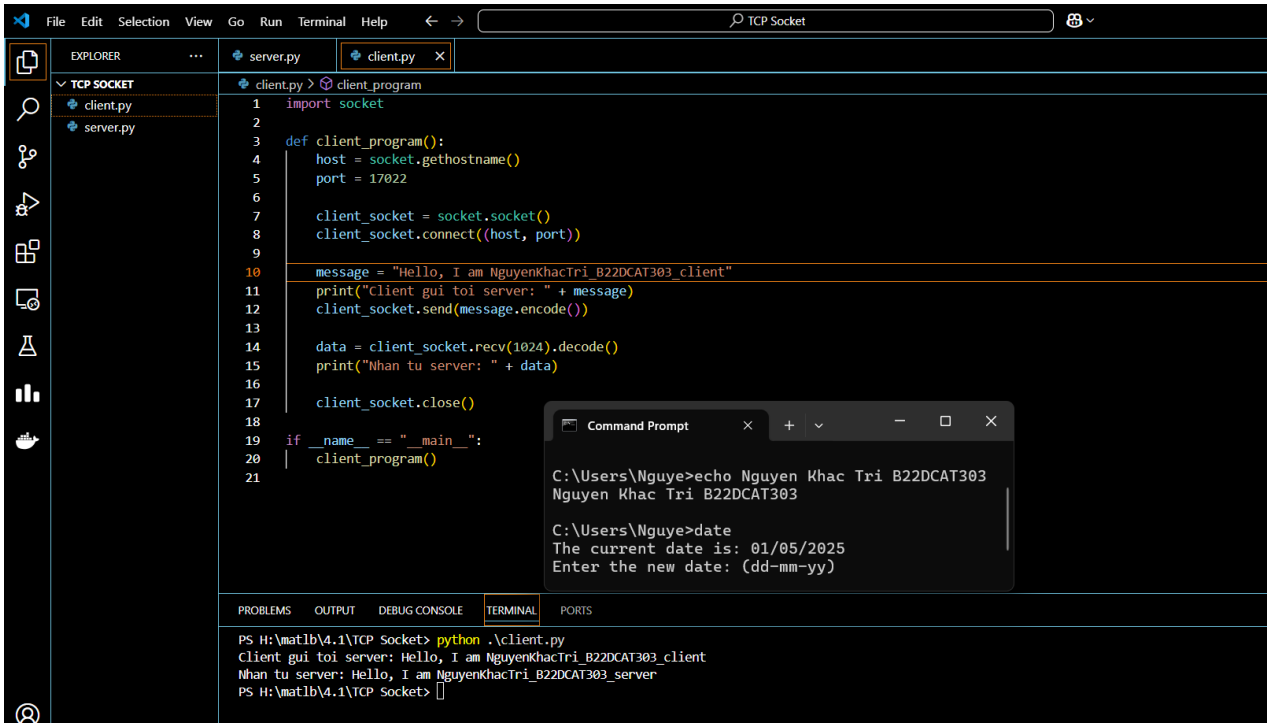
Hình 5 Lập trình server

- Chạy server sau đó chạy client.
- Client gửi thông điệp cá nhân hóa cho server: “Hello, I am NguyenKhacTri_B22DCAT0303_client”.



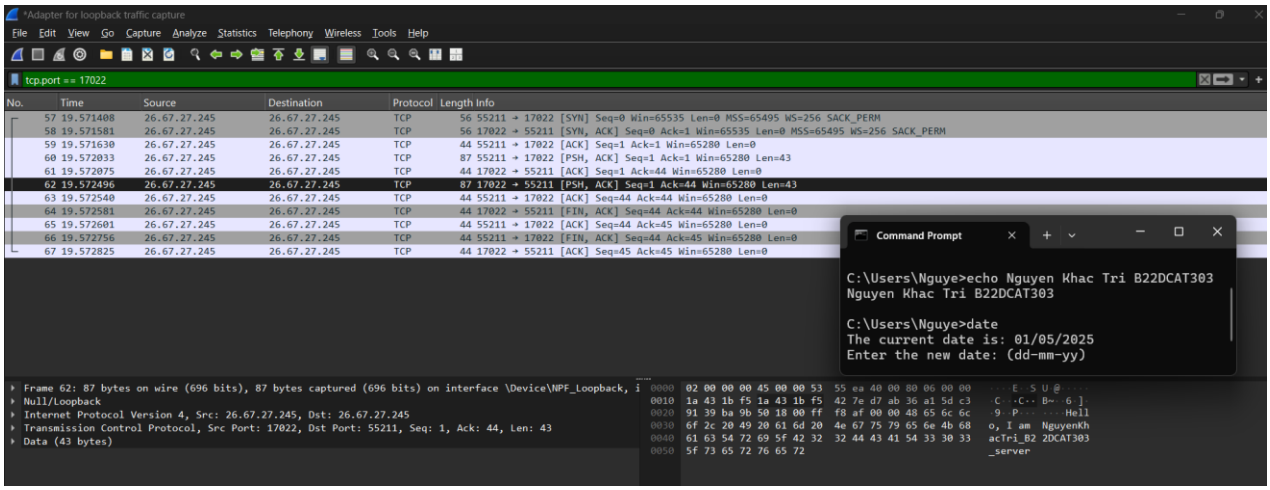
Hình 6 Phía Server sau khi nhận phản hồi từ Client

- Server nhận được hiển thị thông điệp nhận được và gửi lại client thông điệp: Server gửi lại “Hello, I am NguyenKhacTri_B22DCAT0303_server”.

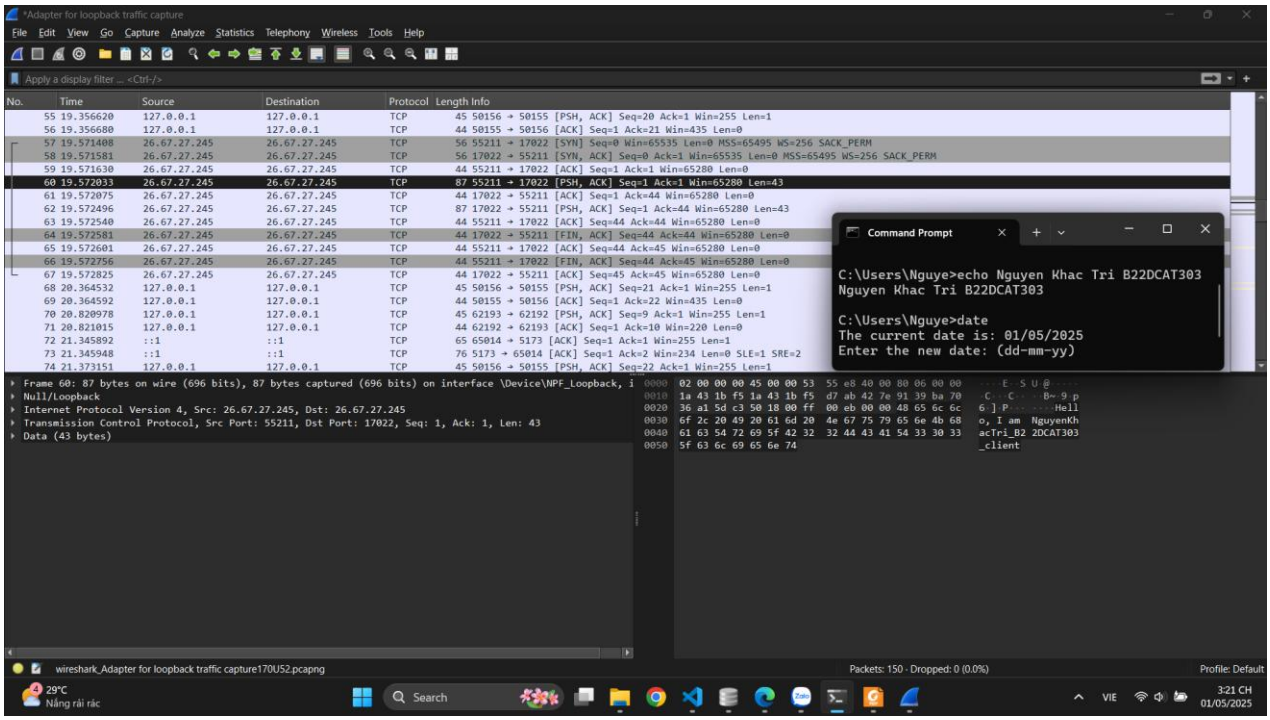


Hình 7 Phía Client sau khi nhận phản hồi từ Server

- Sử dụng Wireshark để bắt các thông tin đã gửi từ client đến server và ngược lại.
- Tiến hành kết nối lại Server với Client và bắt gói tin.



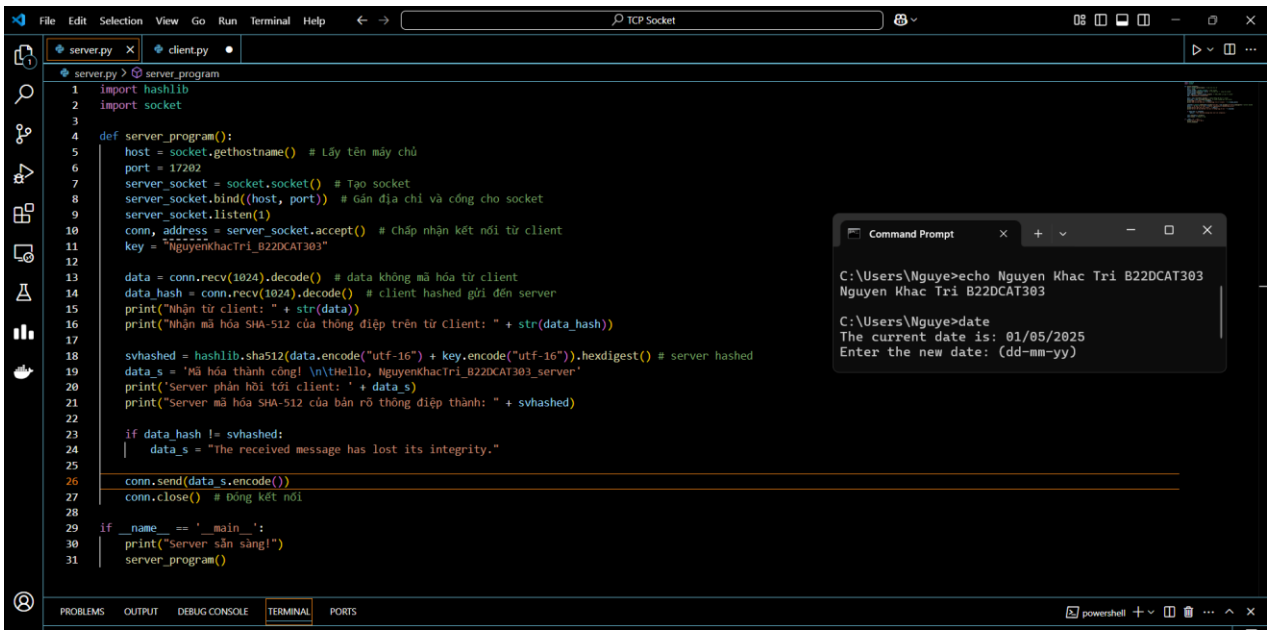
Hình 8 Bắt thông tin được gửi từ Server đến Client



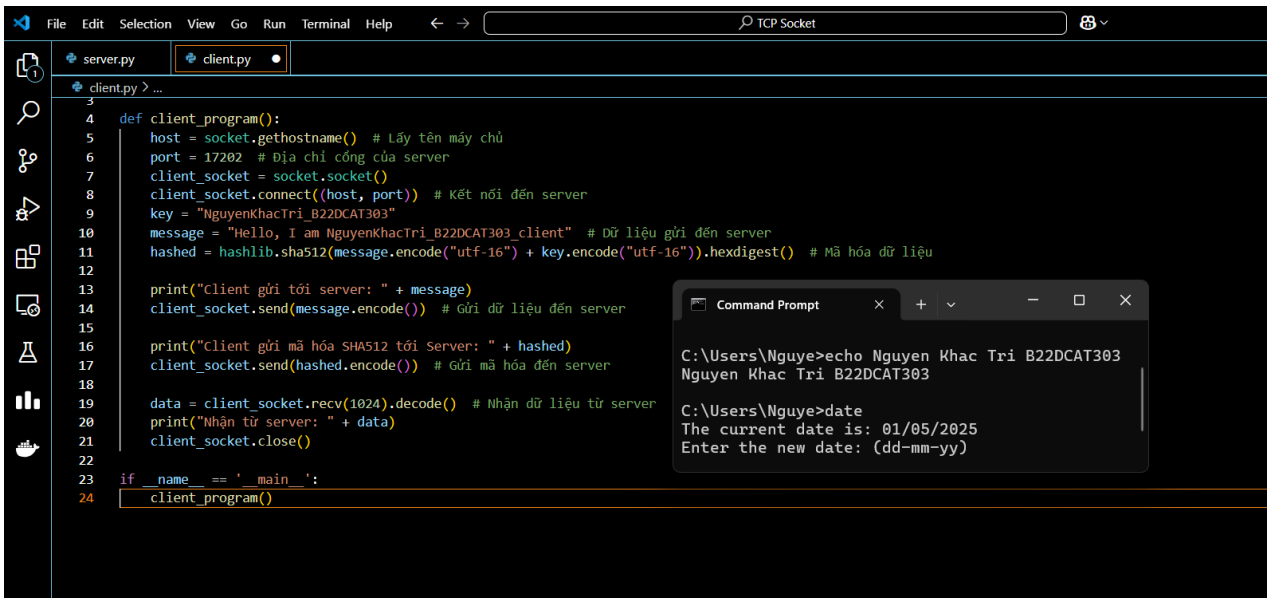
Hình 9 Bắt thông tin được gửi từ Client đến Server

2.2.2 Trao đổi thông điệp giữa client và server và đảm bảo tính toàn vẹn của thông điệp khi trao đổi

Từ client và server, sửa đổi để sao cho: khi gửi thông điệp sẽ gửi kèm theo giá trị băm của (thông điệp+key) để phía bên kia kiểm tra xác minh tính toàn vẹn. Hai bên có thể thống nhất một giá trị key trước đó.

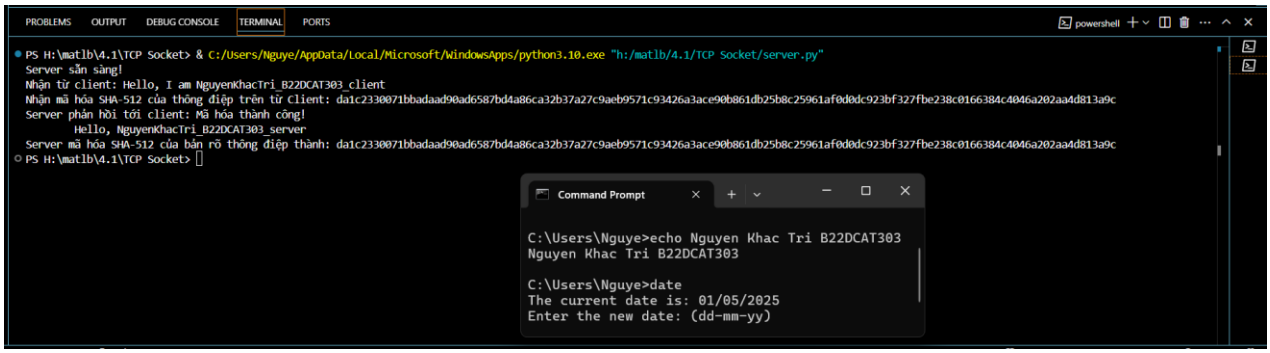


Hình 10 Sửa đổi chương trình bên Server

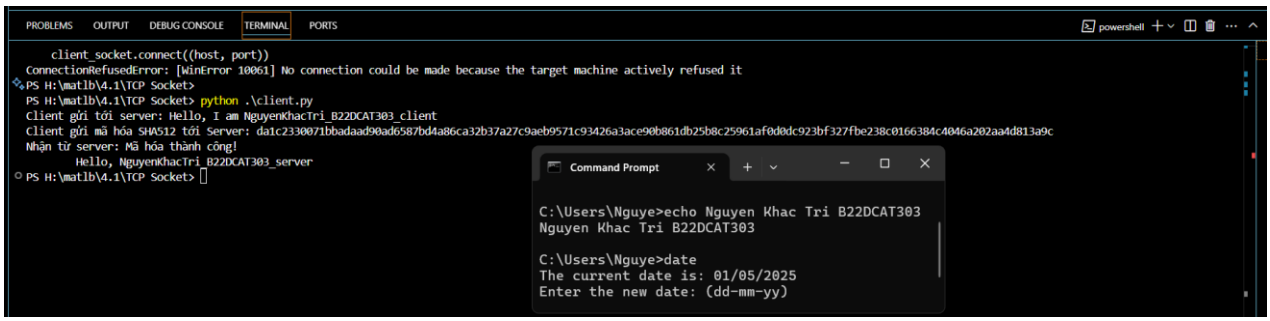


Hình 11 Sửa đổi chương trình bên Client

- Tiến hành chạy lần lượt chương trình bên Server và bên Client để xem kết quả trả về

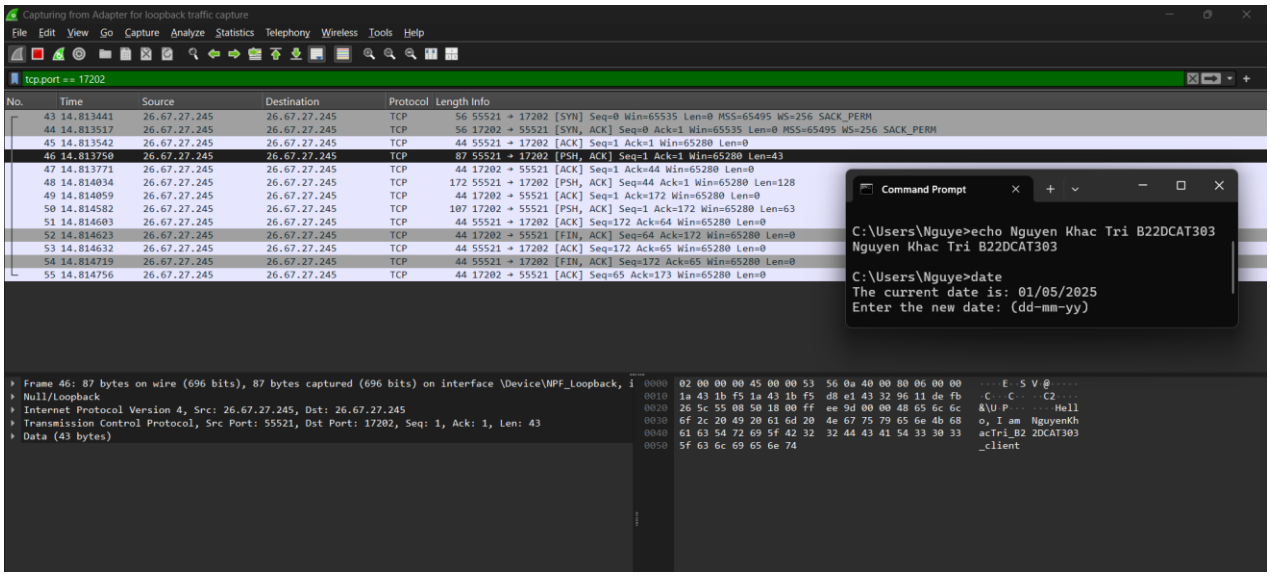


Hình 12 Kết quả trả về bên Server

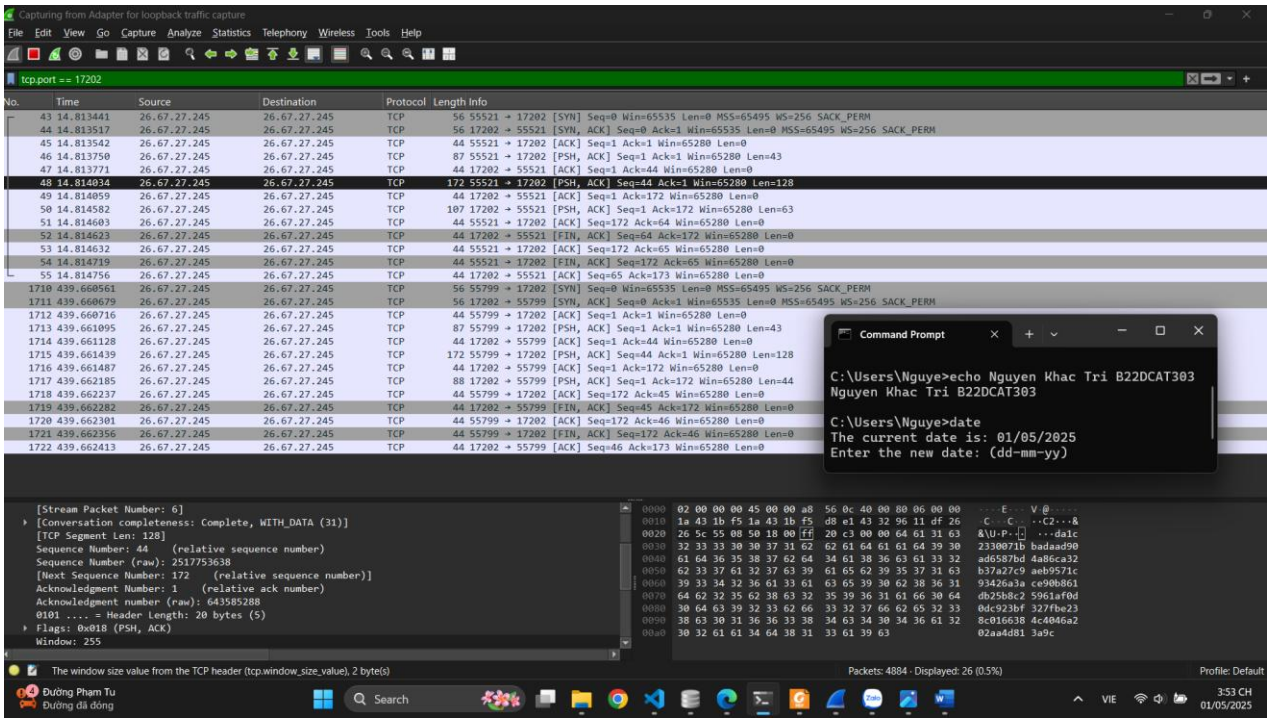


Hình 13 Kết quả khi chạy chương trình bên Client

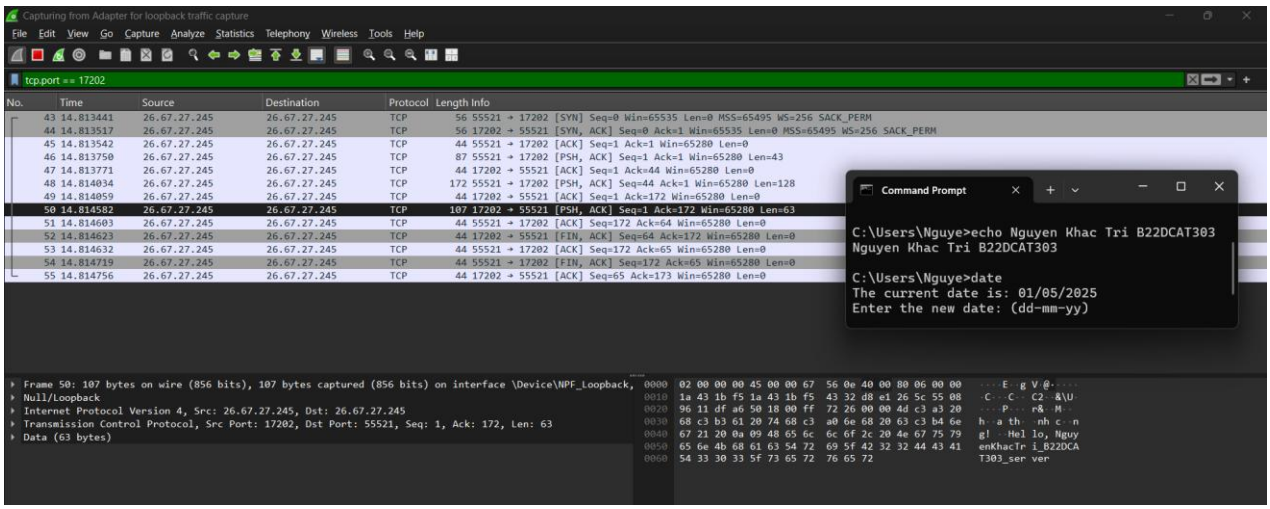
- Bắt các bản tin trao đổi giữa Client và Server trong Wireshark trong trường hợp giống key.



Hình 14 Client gửi thông điệp bản rõ đến Server

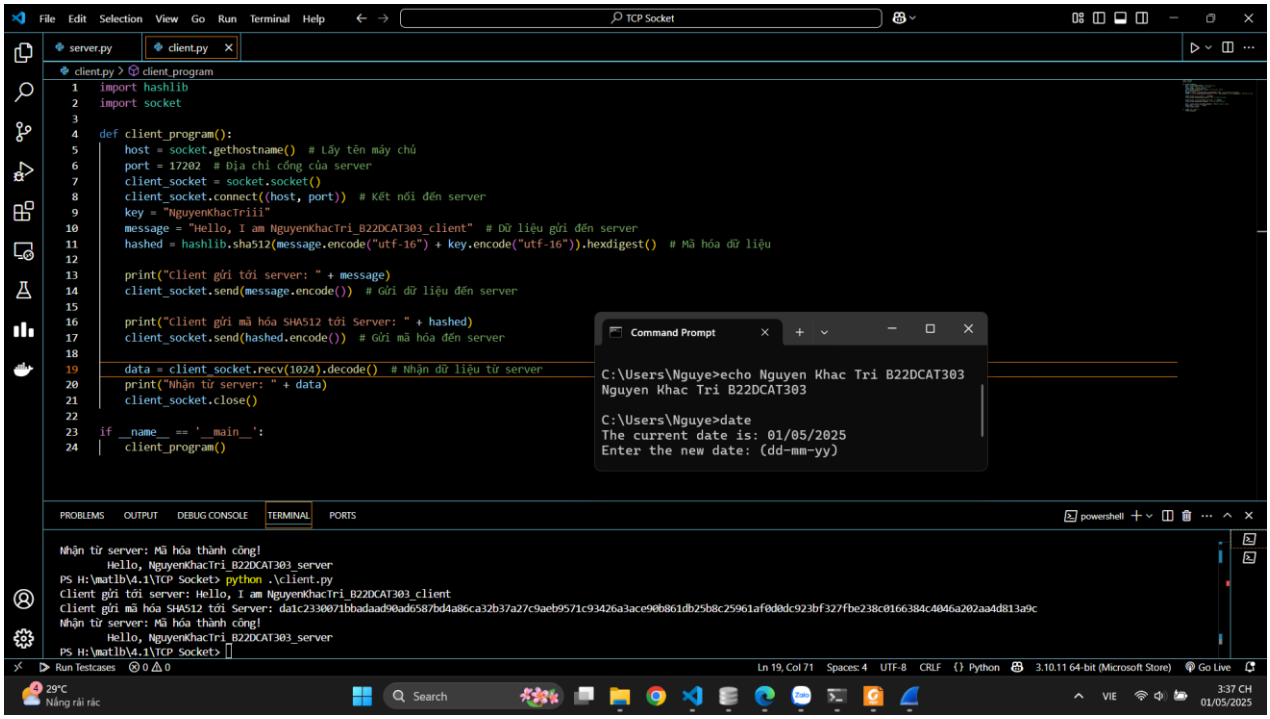


Hình 15 Client gửi thông điệp + key đã được mã hóa SHA512

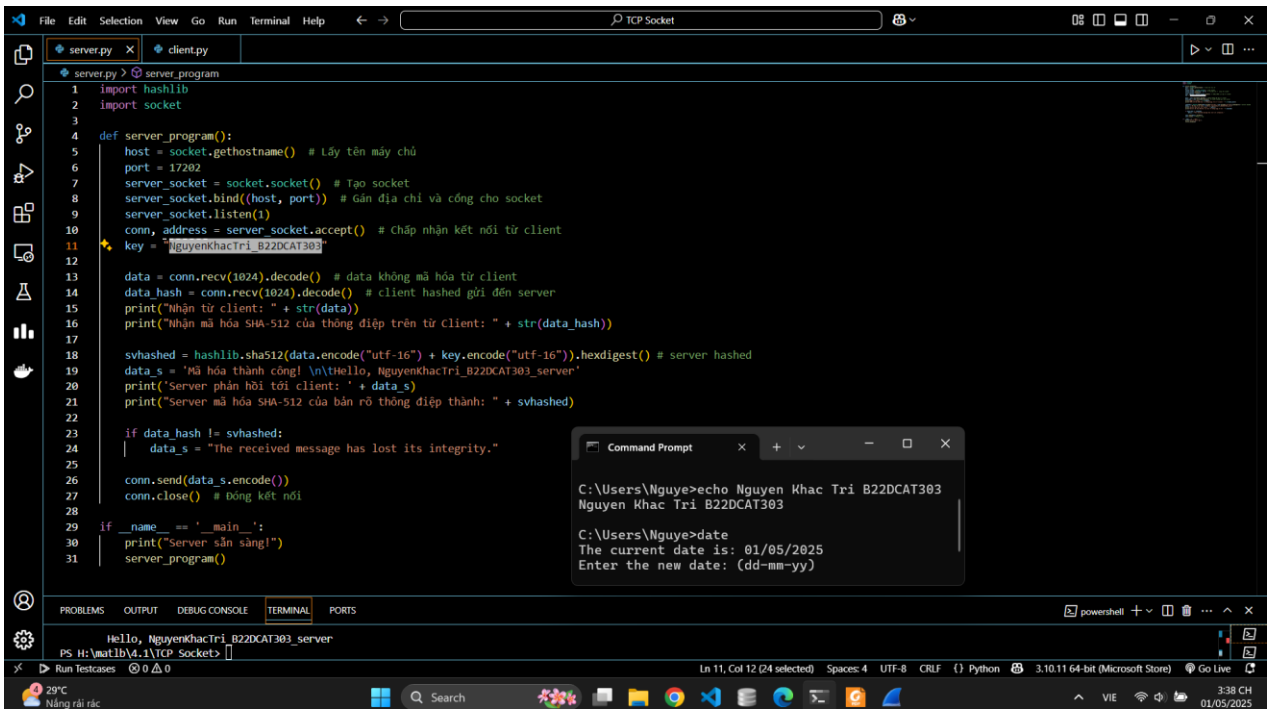


Hình 16 Server gửi phản hồi đến Client sau khi nhận thông điệp + key không thay đổi

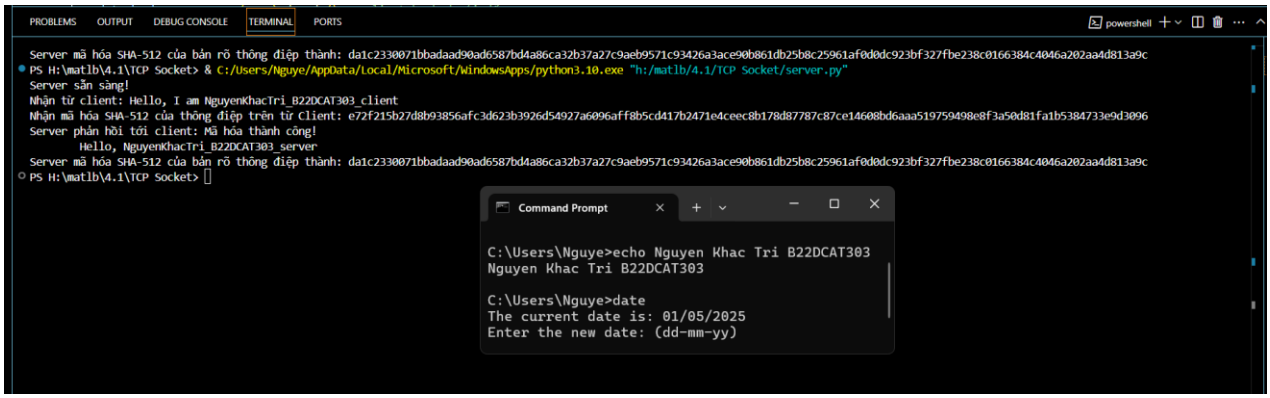
- Thay đổi giá trị key tại client và thực hiện gửi lại, nếu không đáp ứng tính toàn vẹn cần thông báo: “The received message has lost its integrity.”



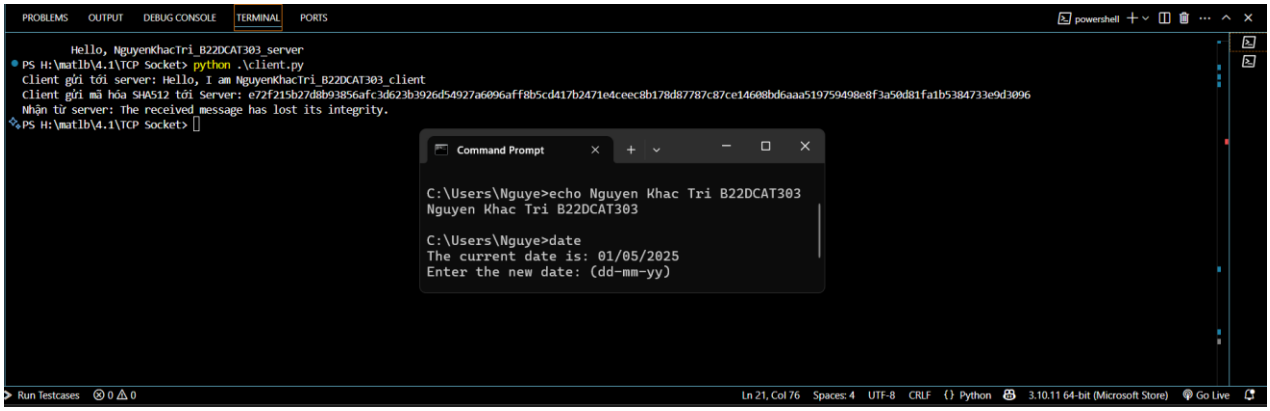
Hình 17 Thay đổi key ở Client → Thông điệp mã hash thay đổi



Hình 18 Bên Server giữ nguyên

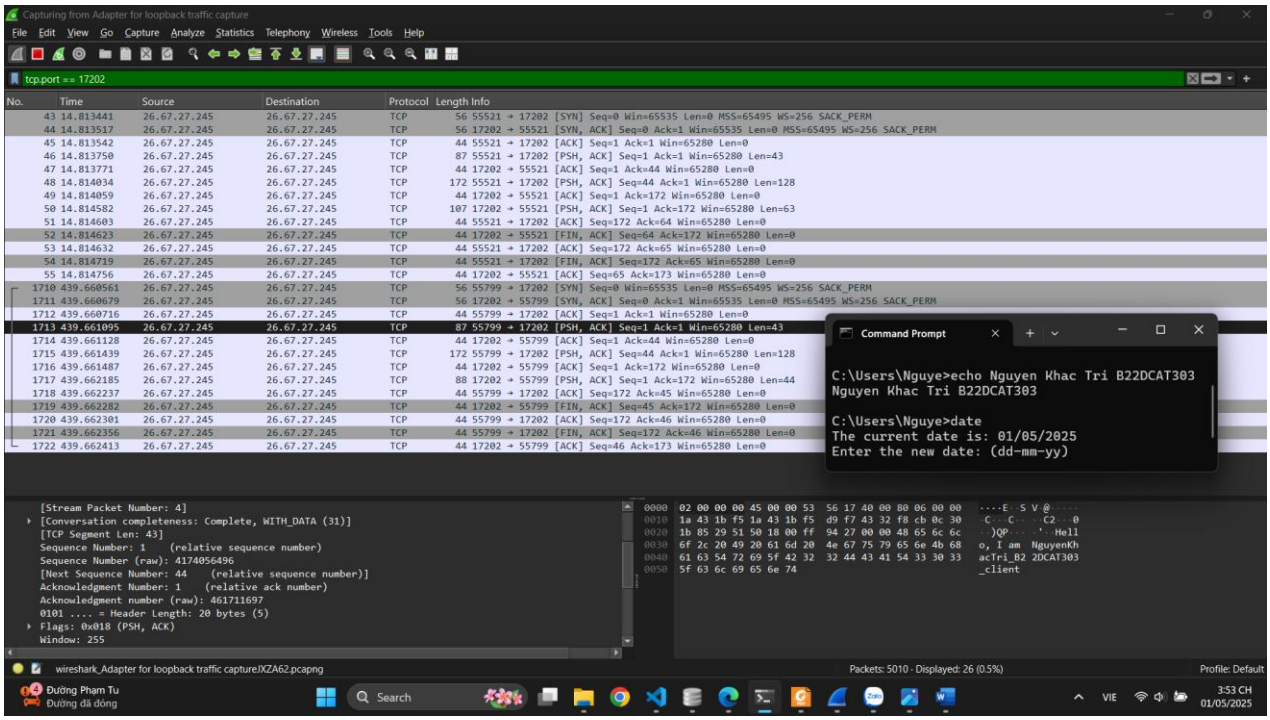


Hình 19 Bên Server: mã hash của thông điệp đã bị thay đổi → Không toàn vẹn dữ liệu

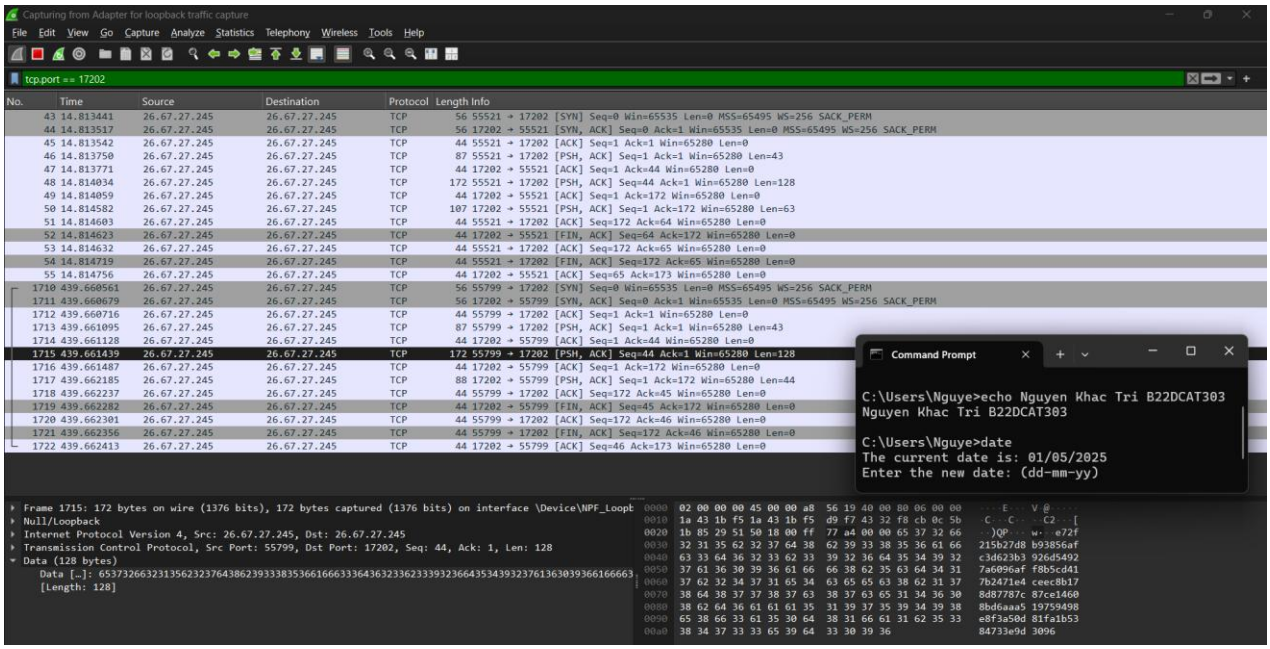


Hình 20 Kết quả, bên Client nhận được thông báo rằng dữ liệu đã bị mất mát

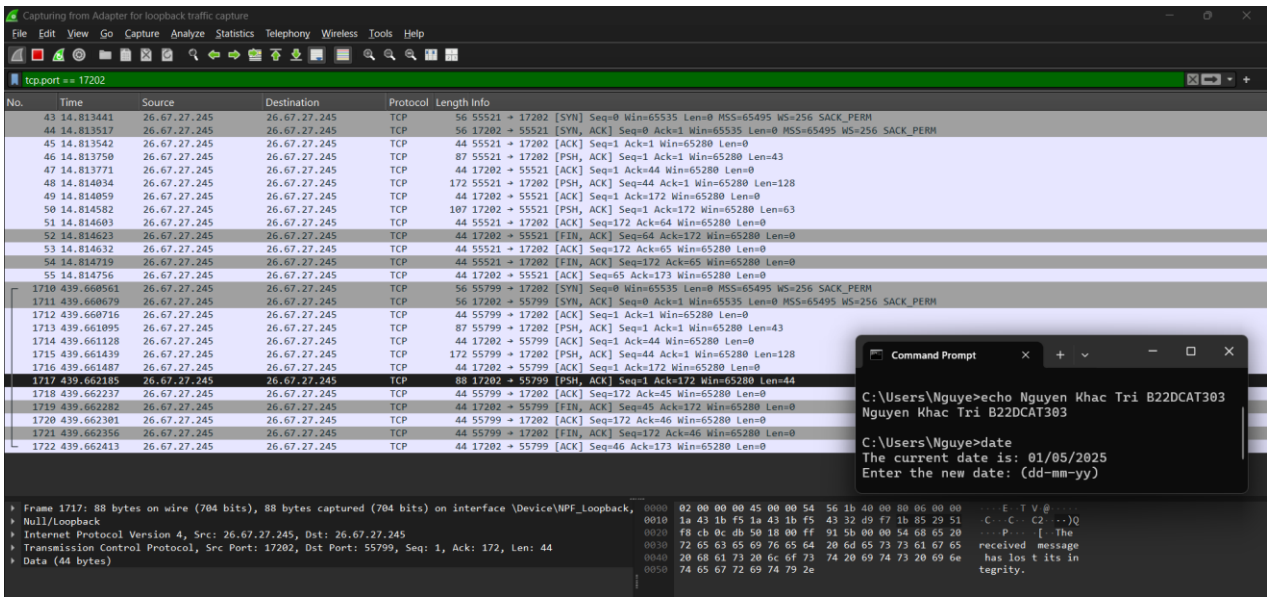
- Bắt được các bản tin trao đổi giữa client và server trong Wireshark trong trương hợp key bị thay đổi.



Hình 21 Thông điệp gửi từ Client đến Server



Hình 22 Khi truyền từ Client → Server, mã hash đã bị thay đổi



Hình 23 Server phản hồi Client: Dữ liệu đã bị mất mát

TÀI LIỆU THAM KHẢO

[1] Chapter 2: Application Layer V8.1 (9/2020) tại địa chỉ http://gaia.cs.umass.edu/kurose_ross/ppt.php (chú ý ví dụ từ trang 105). Một số phiên bản cũ hơn có thể lập trình bằng Java thay vì Python.